Adaptive Spiking Neural Networks for Real-Time Multi-Object Detection Tasks

Donghwa Kang
KAIST
Daejeon, South Korea
anima0729@kaist.ac.kr

Woojin Shin
DeepFusion AI
Seoul, South Korea
wjshin@deep-fusion.com

Cheol-Ho Hong
Chung-Ang University
Seoul, South Korea
cheolhohong@cau.ac.kr

Byunghoon Kang
KAIST
Daejeon, South Korea
brentkang@kaist.ac.kr

Jinkyu Lee*†
Yonsei University
Seoul, South Korea
jinkyu.lee@yonsei.ac.kr

Hyeongboo Baek†
University of Seoul
Seoul, South Korea
hbbaek@uos.ac.kr

Abstract

Providing deterministic *timing guarantees*, beyond merely optimizing the accuracy-latency trade-off, is a critical yet unaddressed challenge for ultra-low-power spiking neural networks (SNNs) in resource-limited, safety-critical systems. We propose RT-SNN, an adaptive SNN method integrating a system-level scheduling framework for multi-object detection. For the *first time*, it co-optimizes inference accuracy while providing strict timing guarantees. RT-SNN orchestrates inference at frame and timestep levels, introducing flexible timestep control and a membrane potential reuse mechanism to enhance accuracy without increasing latency. Evaluations on the KITTI dataset demonstrate RT-SNN significantly improves accuracy and energy efficiency compared to state-of-the-art SNNs and traditional ANNs. Furthermore, a case study on a ROS-based F1/10 autonomous vehicle testbed validates its real-time efficacy for practical deployment.

CCS Concepts

• **Computer systems organization** → **Real-time system architecture**; • **Computing methodologies** → **Object detection**; **Bio-inspired approaches**.

Keywords

Multi-object detection, spiking neural network, timing guarantee

ACM Reference Format:
Donghwa Kang, Woojin Shin, Cheol-Ho Hong, Byunghoon Kang, Jinkyu Lee, and Hyeongboo Baek. 2026. Adaptive Spiking Neural Networks for Real-Time Multi-Object Detection Tasks. In *Proceedings of (DAC'26).* ACM, Long Beach, CA, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

*Jinkyu Lee initiated this work while affiliated with Sungkyunkwan University.
†Hyeongboo Baek and Jinkyu Lee are corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
DAC '26, Long Beach, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804064

1 Introduction

The deployment of perception in safety-critical systems, such as autonomous vehicles (AVs), is governed by a non-negotiable requirement: providing deterministic *timing guarantees* for tasks to complete before their deadlines, in addition to achieving high accuracy under resource constraints. Spiking neural networks (SNNs) have emerged as a promising paradigm for this domain due to their exceptional energy efficiency, derived from sparse, event-driven computation [7, 23, 24]. However, the intrinsic accuracy-latency trade-off in SNNs—where achieving higher accuracy demands more computational timesteps—poses a fundamental obstacle to providing such guarantees. Consequently, current research has remained predominantly model-centric, focusing on optimizing points on this trade-off curve while overlooking the paramount system-level challenge of enforcing safety-critical hard deadlines through co-designed scheduling. This creates a critical gap between the theoretical potential of SNNs and their practical viability in real-world systems, motivating the need for a framework that integrates adaptive SNN inference with real-time scheduling.

We propose RT-SNN, a novel adaptive SNN methodology with a system-level scheduler for SNN-based perception in demanding AV contexts. To our knowledge, RT-SNN is the *first* framework architected to concurrently optimize inference accuracy and provide deterministic timing guarantees for SNN-based MOD tasks. RT-SNN employs a collaborative architecture (e.g., Kapoho Bay [8]) where intensive SNN inference is offloaded to the NPU, while the host CPU manages real-time scheduling. At its core, RT-SNN generates multiple execution *options* using intrinsic SNN properties (e.g., configurable timesteps), enabling fine-grained control over the accuracy-latency trade-off. Its scheduling framework then dynamically selects the option predicted to maximize accuracy at runtime while strictly enforcing hard real-time deadlines and resource constraints.

To validate our approach, we implement RT-SNN with Spiking-YOLO [15], an ANN-to-SNN converted model, and benchmark its performance against both SNN and ANN baselines. To ensure a fair comparison, we augment the baseline SNNs, which lack formal timing guarantees, with an offline schedulability analysis. On the KITTI dataset, RT-SNN demonstrates significant improvements, achieving up to 306.7% higher accuracy than existing SNNs at the same energy budget and requiring up to 21.9% less energy at equivalent accuracy. Furthermore, RT-SNN consumes up to 79.0% less

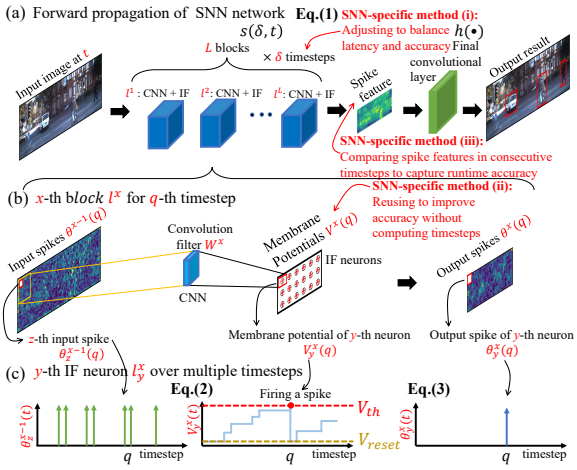


Figure 1: Spiking neural network architecture for multi-object detection (e.g., Spiking-YOLO), illustrating how membrane potentials accumulate over timesteps. The diagram highlights where RT-SNN's key methods— (i) flexible timestep scheduling, (ii) membrane potential reuse, and (iii) runtime accuracy capture—are integrated into the inference pipeline.

energy than ANN baselines at a comparable accuracy level. Finally, a case study on a ROS-based F1/10 autonomous driving testbed confirms the practical efficacy of RT-SNN, showcasing its ability to detect and respond to obstacles under strict real-time constraints.

Contribution. We articulate our novel contributions compared to related work as follows. To our knowledge, RT-SNN is the *first* adaptive SNN designed to provide timing guarantees in real-time perception systems. It pioneers a runtime accuracy prediction mechanism for complex SNN-based MOD, a capability absent in prior MOD works [6, 9, 15, 26] and previously limited to simpler classification tasks [17, 18, 24]. Furthermore, we introduce membrane potential reuse (MPR), a novel, SNN-specific technique that enhances accuracy without extending computational timesteps. Finally, the RT-SNN framework is designed to be *model-agnostic* and is therefore applicable to most (if not all) state-of-the-art (SOTA) SNN models [15, 26].

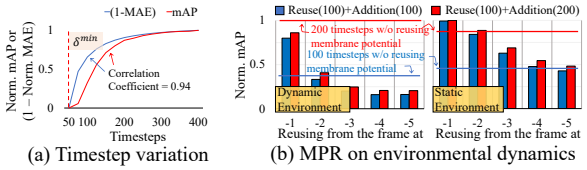
2 Background

Given an input frame t and timesteps δ, SNN-based MOD is defined by $f_\delta(t) = h(s(\delta, t))$, where $s(\delta, t)$ represents the spike feature [29] extracted across timesteps, and $h(\cdot)$ is the final convolutional layer. The forward propagation follows:

$$f_\delta(t) = h(s(\delta,t)) = h\Big(\frac{1}{\delta}\sum_{i=1}^{\delta} l^L \circ l^{L-1} \circ \cdots \circ l^1(t)\Big), \tag{1}$$

where l^x is the x-th block in the network hierarchy ($1 \le x \le L$). As shown in Fig. 1(a), the SNN processes input through L blocks per frame in each timestep, repeating δ times to extract spike features, which are then processed by $h(\cdot)$ for detection.

Fig. 1(b) illustrates the computational flow through convolutional and integrate-and-fire (IF) neuron [4] layers of block l^x at timestep q ($1 \le q \le \delta$). Within l^x, the y-th IF neuron updates membrane



(a) Timestep variation (b) MPR on environmental dynamics

Figure 2: Underlying principles of RT-SNN experimented on the KITTI dataset. (a) The inverse of MAE between spike features strongly correlates with the final mAP. (b) The effectiveness of MPR depends on environmental dynamics; for example, reusing potentials from older frames ($t-3$ to $t-5$) is beneficial in static environments but detrimental in dynamics.

potential $V_y^x(q)$ by convolving spike inputs $\Theta^{x-1}(q)$ from block x-1[1] with convolutional weights W^x and adding the previous potential $V_y^x(q\text{-}1)$:

$$V_y^x(q) = V_y^x(q-1) + \sum_z W_{y,z}^x \Theta_z^{x-1}(q) + b_y^x, \tag{2}$$

where $W_{y,z}^x$ represents convolution weights and b_y^x is the bias. If $V_y^x(q)$ exceeds threshold V_{th}, the neuron emits a spike $\Theta_y^x(q)$ and resets its potential:

$$\Theta_y^x(q) = \begin{cases} 1 & \text{if } V_y^x(q) \ge V_{th} \\ 0 & \text{otherwise} \end{cases}. \tag{3}$$

The generated spikes propagate to subsequent layers if applicable. Fig. 1(c) visualizes the y-th IF neuron's membrane potential accumulation and spike generation over δ timesteps.

Among SNN training methods [15, 16, 22], we adopted ANN-to-SNN conversion compared to direct training because it is the preferred approach for applications, like ours, that must maximize accuracy under constrained computing resources due to its superior performance.

3 Overall Architecture

The design of RT-SNN relies on novel mechanisms for runtime SNN performance estimation and optimization. Since existing accuracy estimation methods like cross-entropy [18] are limited to classification, we propose using the mean absolute error (MAE) between spike features. As validated in Fig. 4 and Fig. 2(a), MAE serves as a lightweight proxy for the final mAP, exhibiting a strong Pearson correlation of 0.95. Second, we introduce MPR, a technique that enhances efficiency by transferring membrane potentials from previous frames, thereby reducing the computational warm-up phase. However, uninformed MPR in dynamic scenes can degrade performance (Fig. 2(b)). To address this, we define *membrane confidence*, a predictive metric quantifying the expected accuracy of applying MPR. These principles drive a framework that dynamically adapts timesteps and selective MPR to maximize accuracy under real-time constraints.

The overall architecture of RT-SNN, depicted in Fig. 3, operationalizes these principles through the interaction of three concurrent components: a *frame-level scheduler*, a *dynamic timestep execution pipeline*, and a *membrane confidence estimator*. The scheduler makes high-level decisions (e.g., the number of timesteps with

[1]For l^1, input images are encoded to spikes $\Theta^1(q)$ via IF neurons [27].

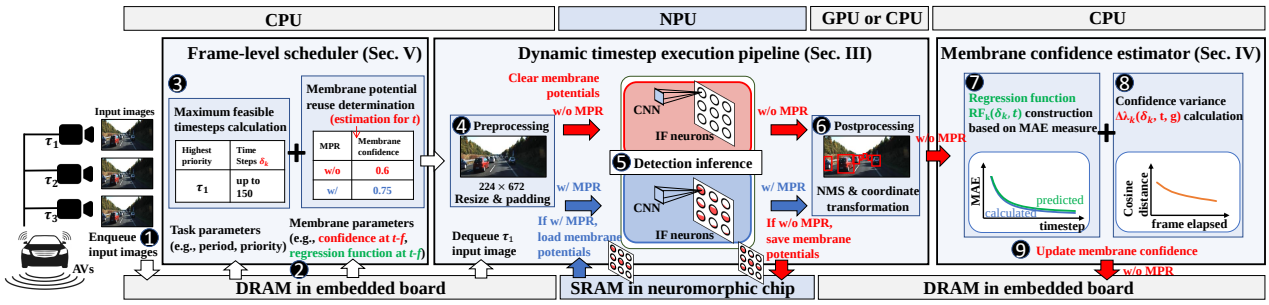


Figure 3: Overall architecture of the RT-SNN framework. Targeting a CPU-NPU heterogeneous platform, the system coordinates via shared memory. On the CPU, the *frame-level scheduler* receives input images (❶) and, using parameters from previous frames (❷), determines the maximum feasible (i.e., no deadline miss guarantee) timesteps and whether to apply MPR (❸). This decision is guided by the *membrane confidence estimator*, which constructs an MAE-based regression model (❼) and calculates confidence variance using cosine similarity between recent spike features (❽) to predict future accuracy. On the NPU, the *dynamic timestep execution pipeline* preprocesses the image (❹) and performs SNN inference (❺). Depending on the scheduler's decision, it either resets the neuron potentials or loads them (i.e., extracted at earlier frames) from memory for MPR. After inference and postprocessing (❻), the results are used to update the membrane confidence (❾) for the next cycle.

or without MPR), which are then executed by the inference pipeline. The confidence estimator provides the predictive feedback necessary for the scheduler to make informed choices. The detailed mechanisms of our membrane confidence metric and the confidence-aware scheduling algorithm are elaborated in Secs. 4 and 5, respectively.

4 Membrane confidence

To empower the scheduler to select the optimal execution option (i.e., flexible timesteps with or without MPR), this section introduces *membrane confidence*, a novel metric for predicting the inference accuracy of an SNN task at runtime. This metric is built upon MAE, which efficiently measures the variation between sequential, lightweight spike features. For a task τ_k at frame t with δ_k timesteps and a comparison interval g, the MAE, $M_k(\delta_k, t, g)$, is formulated as:

$$M_k(\delta_k, t, g) = |\hat{s}_k(\delta_k, t) - \hat{s}_k(\delta_k - g, t)|. \tag{4}$$

Using MAE, we define the membrane confidence $\lambda_k(\delta_k, t, g)$ as a normalized score from 0 to 1 that quantifies the progress toward a predefined MAE threshold (M_{th}), which corresponds to maximum accuracy. It is calculated as:

$$\lambda_k(\delta_k, t, g) = 1 - \max\left(\frac{M_k(\delta_k, t, g) - M_{th}}{M_k(\delta^{min} - g, t, g) - M_{th}}, 0\right), \tag{5}$$

where δ^{min} is the minimum timesteps guaranteed by the offline schedulability analysis (Sec. 5). Since the scheduler must decide on an execution strategy before inference, it is essential to accurately estimate this confidence value beforehand. The following subsections detail how we estimate membrane confidence for executions without and with MPR.

4.1 Confidence estimation without MPR

To estimate the membrane confidence for the current frame t without MPR, we leverage the MAE measurements from the most recent non-MPR execution at frame t-f. As shown in Fig. 2(a), the MAE curve resembles a rational function. We thus model it using a regression function, $RF_k(x, t)$, which takes a timestep x as input

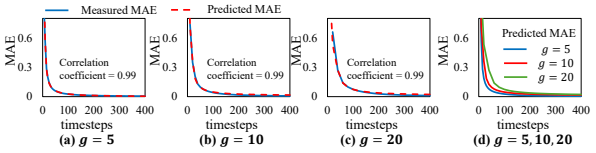


Figure 4: Predicted versus actual MAE curves validating RT-SNN's runtime accuracy estimation using regression models across various spike feature intervals g = 5, 10, and 20 on the KITTI dataset.

and outputs an estimated MAE:

$$RF_k(x, t) = \frac{\alpha}{x} + \beta, \tag{6}$$

where parameters α and β are optimized using the Levenberg-Marquardt method [28] on the historical data from frame t-f. As validated in Fig. 4, this regression model achieves a high correlation (0.99) with the actual MAE. Using this function, we can predict the membrane confidence for any given number of timesteps δ_k as:

$$\bar{\lambda}_k(\delta_k, t, g) = 1 - \max\left(\frac{RF_k(\delta_k, t) - M_{th}}{RF_k(\delta^{min} - g, t) - M_{th}}, 0\right). \tag{7}$$

4.2 Confidence estimation with MPR

Estimating confidence for an execution with MPR is more complex, as it must account for three factors: (i) the initial confidence of the reused membrane potentials from frame t-f, (ii) the decay in confidence over f frames due to environmental dynamics, and (iii) the new confidence gained from additional timesteps at frame t. Fig. 5 illustrates this computation.

To address (ii), we quantify the environmental dynamics by calculating a *membrane confidence variance*, $\Delta\lambda_k$, between the two most recent non-MPR frames (t-h and t-f, where $h > f$). This variance is derived from the *cosine distance* [1] between their spike features:

$$\Delta\lambda_k(\delta_k^*, h, f) = \frac{\gamma}{h - f} \cdot \left(1 - \max\left(\frac{\hat{s}(\delta_k^*, t-h) \cdot \hat{s}(\delta_k^*, t-f)}{|\hat{s}(\delta_k^*, t-h)||\hat{s}(\delta_k^*, t-f)|}, 0\right)\right), \tag{8}$$

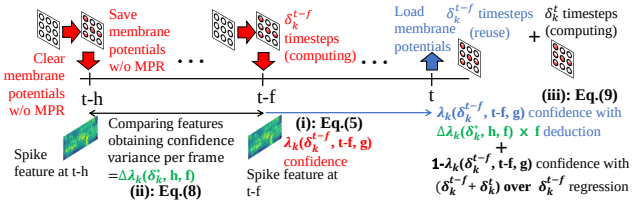


Figure 5: Computation of membrane confidence for guiding effective MPR, based on MAE thresholds relative to minimum timestep baselines.

where δ_k^* is the lesser of the timesteps used at t-h and t-f, and $\gamma \geq 0$ is a sensitivity parameter. If a past frame at t-h is unavailable, the variance is estimated as $1/f$.

Finally, to address (iii), we formulate the estimated confidence with MPR, $\overline{\lambda}_k^+$, as a weighted sum that combines the decayed past confidence with the newly generated confidence:

$$\overline{\lambda}_k^+(\delta_k, t, g) = \lambda_k^{t-f} \cdot \max\left(1 - \Delta\lambda_k(\delta_k^*, h, f) \times f, 0\right) + \left(1 - \lambda_k^{t-f}\right) \cdot \left(1 - \max\left(\frac{RF_k(\delta_k^{t-f} + \delta_k, t) - M_{th}}{RF_k(\delta_k^{t-f}, t) - M_{th}}, 0\right)\right), \quad (9)$$

where λ_k^{t-f} is the original confidence from frame t-f. The first term represents the decayed confidence from the reused potentials. The second term, weighted by the remaining confidence gap $(1 - \lambda_k^{t-f})$, represents the new confidence gained by executing an additional δ_k timesteps, as estimated by our regression function (Eq. 6).

5 Scheduling Framework for RT-SNN

We model the system as a set of n periodic SNN tasks, $\tau = \{\tau_1, \ldots, \tau_n\}$, where each task τ_i is defined by a period T_i (which is also its relative deadline) and a worst-case execution time (WCET), $C_i(\delta)$. The WCET is linearly dependent on the number of inference timesteps, δ, and is formulated as $C_i(\delta) = c_i^{sche} + \delta \cdot c_i^{ts} + c_i^{post} + c_i^{conf}$. This includes the overhead for scheduling (c_i^{sche}), per-timestep inference on the NPU (c_i^{ts}, set to $1ms$ [21]), post-processing (c_i^{post}), and confidence estimation (c_i^{conf}), along with all associated data communication costs. Each job of task τ_i is released every T_i and must complete before its deadline.

RT-SNN is built upon a baseline non-preemptive, fixed-priority (NPFP) scheduling policy. To provide initial timing guarantees, we define NPFP^{min} (NPFP with MINimum execution) [14], where every job of a task τ_i runs for a minimum duration $C_i(\delta^{min})$ necessary to achieve a baseline accuracy. The schedulability of this approach is guaranteed if the condition in Eq. (10) holds for all tasks, as established by the following lemma.

Lemma 1. *Suppose an SNN task set τ is scheduled by NPFP^{min}. If every $\tau_i \in \tau$ satisfies Eq.* (10)*, then no job will miss its deadline.*

$$C_i(\delta^{min}) + \max_{\tau_j \in LP(\tau_i)} C_j(\delta^{min}) + \sum_{\tau_h \in HP(\tau_i)} \left\lceil \frac{T_i + T_h - C_h(\delta^{min})}{T_h} \right\rceil \cdot C_h(\delta^{min}) \leq T_i. \quad (10)$$

Proof. This condition is derived from standard response time analysis for non-preemptive systems [3, 11]. The three terms on the left-hand side of Eq. (10) represent the task's own execution time, the maximum possible blocking from any lower-priority task, and the maximum interference from all higher-priority tasks within the interval T_i, respectively. If their sum does not exceed the deadline T_i, the task is schedulable. □

Algorithm 1 The NPFP^{mem} scheduling algorithm

At t, at which the highest-priority active job of τ_k starts:

1: Let χ be the set of tasks whose schedulability is affected by τ_k's execution before its deadline $r_k^{(t)}$.
2: $\Delta\delta = \left(r_k^{(t)} - t - C_k(\delta^{min})\right)/c_k^{ts}$
3: **for** $\tau_i \in \chi$ **do**
4: $\quad \Delta\delta = \min(\Delta\delta_i^{(t)}, \Delta\delta)$
5: **end for**
6: **if** $\overline{\lambda}_k(\delta^{min} + \Delta\delta, t, g) \geq \overline{\lambda}_k^+(\delta^{min} + \Delta\delta, t, g)$ **then**
7: $\quad$ Execute τ_k for $C_k(\delta^{min} + \Delta\delta)$
8: **else**
9: $\quad$ Execute τ_k for $C_k(\delta^{min} + \Delta\delta)$ with MPR
10: **end if**
11: **for** $\tau_i \in \chi$ **do**
12: $\quad \Delta\delta_i^{(t)} = \Delta\delta_i^{(t)} - \Delta\delta$
13: **end for**
14: Reset $\Delta\delta_k^{(t)}$ based on the slack defined in Eq. (10).

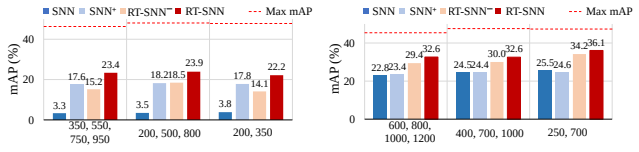
While NPFP^{min} ensures timeliness, it cannot maximize accuracy. To address this, we propose NPFP^{mem} (NPFP with MEMbrane confidence), our dynamic scheduling algorithm detailed in Algo. 1. NPFP^{mem} operates on two principles: (P1) it strictly adheres to the offline timing guarantees established by Lemma 1, and (P2) it utilizes any available system slack to maximize inference accuracy at runtime, guided by the membrane confidence metrics from Sec. 4.

The core strategy of NPFP^{mem} is as follows. When a task τ_k is scheduled to run, the algorithm first calculates the maximum number of additional timesteps, or *slack* ($\Delta\delta$), that τ_k can execute without causing any task in the system to miss its deadline (Lines 1–5). It then uses the estimated confidence metrics, $\overline{\lambda}_k$ and $\overline{\lambda}_k^+$, to predict which execution option—running for $\delta^{min} + \Delta\delta$ timesteps with or without MPR—will yield higher accuracy (Line 6). The algorithm commits to the superior option, thereby converting idle time into improved performance (Lines 7–10). Finally, it updates the slack budgets for all affected tasks to maintain system-wide schedulability for future decisions (Lines 11–14).

Theorem 1. *Suppose that an SNN task set τ is scheduled by NPFP^{mem} and satisfies Eq.* (10)*. The following two properties hold: (i) offline timing guarantees: no job of any task $\tau_i \in \tau$ will miss its deadline, and (ii) runtime accuracy improvement: the accuracy of every job is greater than or equal to its accuracy under NPFP^{min}.*

Proof. Timing guarantees hold because slack execution is only permitted if it does not violate the schedulability condition for any task (Lines 2–5). Accuracy is improved because the number of executed timesteps is $\delta^{min} + \Delta\delta \geq \delta^{min}$ (Lines 7 and 9). □

Run-time complexity. The scheduling decision for NPFP^{mem} at any time t requires calculating $\Delta\delta$, which involves iterating through the set of affected tasks χ. Since the number of iterations in Lines 3–5 and 11–13 is $|\chi|$, the complexity is $O(|\chi|)$, ensuring minimal overhead.



(a) For δ^{min}=80 (b) For δ^{min}=120

Figure 6: Experiment results on existing SNN with RT-SNN and its variations.

6 Evaluation

Hardware and software. Experiments were conducted on a server equipped with AMD EPYC 7302 16-Core processors, 198GB RAM, and an NVIDIA RTX A6000 GPU. We employ an SNN simulator aligned with the Spiking-YOLO environment [15]. The detection model used is Spiking-YOLO, trained on the MS COCO dataset [19], with performance evaluated on the KITTI dataset [10] using mAP metric [5]. RT-SNN supports any fixed-priority scheduling; we employed the well-known rate-monotonic (RM) policy [20], which assigns higher priorities to tasks with shorter periods. The WCET values for the components of the RT-SNN scheduling framework, presented in Table 1, were determined by measuring the execution time of each component 1000 times. Note that the per-timestep inference time (c_i^{ts}), nominally representing NPU computation, is simulated on the GPU in our environment; its WCET is assumed to be 1 *ms*, consistent with widely accepted values in prior studies [2, 13, 15] for a pair comparision.

Considered approaches. To evaluate performance, we compare the following approaches.

- **SNN**: NPFP^{min} (i.e., w/o MPR) with the largest δ^{min} that satisfies Eq. (10). Note that this operates identically to the existing SNN (e.g., Spiking-YOLO) but is extended to satisfy Eq. (10) for timing guarantees.
- **SNN$^+$**: SNN w/ MPR for every two frames.
- **RT-SNN**: NPFP^{mem} (in Algo. 1, i.e., w/ MPR) with δ^{min}=50 (our ultimate approach).
- **RT-SNN$^-$**: RT-SNN w/o MPR.

For a favorable comparison with SNN and SNN$^+$, we assigned the largest δ^{min} (subject to Eq. (10)) to SNN and SNN$^+$, as a δ^{min} of 50 yielded much lower accuracy.

6.1 Comparison with existing SNN

General accuracy. As all considered approaches satisfy timing guarantees, we then evaluate the accuracy of the considered approaches for n ranging from 2 to 4 with different periods (denoted by the x-axis in each figure), setting g=10, γ=3 and M_{th}=0.02 as experiment parameters. Fig. 6(a) presents the accuracy for both SNN and SNN$^+$ at δ^{min}=80 (and for RT-SNN$^-$ and RT-SNN at δ^{min}=50 as a default value). Note that Max mAP is the result obtained from running SNN for 400 timesteps without accounting for timing guarantees. SNN records the least accuracy due to limited timesteps, while SNN$^+$ shows some improvement by alternating MPR. By leveraging runtime slack ($\Delta\delta$), RT-SNN$^-$ surpasses both baselines, and RT-SNN achieves the highest accuracy with its adaptive MPR. Fig. 6(b) reveals that with δ^{min}=120, SNN$^+$ shows reduced accuracy compared to SNN, suggesting that MPR without membrane confidence awareness can lower accuracy at higher timesteps. Conversely, RT-SNN attains superior accuracy by dynamically reusing membrane potentials as needed, guided by membrane confidence.

Table 1: Measured execution times

Device	Hardware	CPU	NPU	GPU	CPU
A6000	Component	c_i^{sche} (≤ 8 tasks)	c_i^{ts} (per timestep)	c_i^{post}	c_i^{conf} (≤ 400 timesteps)
	Average	0.18*ms*	-	2.1*ms*	0.43*ms*
	WCET	**0.3*ms***	**1*ms***	**2.5*ms***	**1.7*ms***
TX2	Component	c_i^{sche} (≤ 2 tasks)	c_i^{ts} (per timestep)	c_i^{post}	c_i^{conf} (≤ 200 timesteps)
	Average	0.67*ms*	-	9.9*ms*	4.9*ms*
	WCET	**2.3*ms***	**1*ms***	**19.8*ms***	**14.9*ms***

Table 2: Comparison of accuracy and energy efficiency among SNN approaches.

Period set (ms)	Approach	Avg. δ_k	mAP (%)	Energy
180, 400	RT-SNN	92.4	**21.8** (/14.4 = **+51.4%**)	**1x** (/1.13x = **-11.5%**)
	SNN$_E$	101	14.4	**1x**
	SNN$_A$	113	**21.5**	1.13x
200, 700	RT-SNN	102	**26.4** (/21.0 = **+25.7%**)	**1x** (/1.11x = **-9.9%**)
	SNN$_E$	111	21.0	**1x**
	SNN$_A$	122	**26.4**	1.11x
400, 450, 500, 550	RT-SNN	68.7	**12.2** (/3.0 = **+306.7%**)	**1x** (/1.28x = **-21.9%**)
	SNN$_E$	77	3.0	**1x**
	SNN$_A$	96	**12.1**	1.28x
450, 500, 550, 600	RT-SNN	73.9	**14.3** (/5.0 = **+186.0%**)	**1x** (/1.24x = **-19.4%**)
	SNN$_E$	81	5.0	**1x**
	SNN$_A$	100	**14.0**	1.24x
650, 750, 850, 950, 1050, 1150, 1250, 1350	RT-SNN	74.5	**13.0** (/4.1 = **+217.1%**)	**1x** (/1.23x = **-18.7%**)
	SNN$_E$	82	4.1	**1x**
	SNN$_A$	100	**12.4**	1.23x

Table 3: Comparison of accuracy and energy efficiency between ANN and SNN.

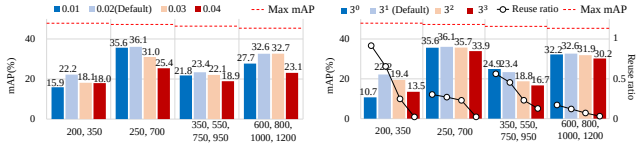
Period set (ms)	Approach	Avg. δ_k	mAP (%)	Energy
180, 400	RT-SNN	92.4	**21.9**	**1x** (/4.09x = **-75.6%**)
	ANN	-	20.8	4.09x
200, 700	RT-SNN	102	**26.4**	**1x** (/4.77x = **-79.0%**)
	ANN	-	27.1	4.77x
400, 450, 500, 550	RT-SNN	68.7	**12.2**	**1x** (/2.23x = **-55.2%**)
	ANN	-	10.0	2.23x
450, 500, 550, 600	RT-SNN	73.9	**14.3**	**1x** (/2.45x = **-59.2%**)
	ANN	-	14.8	2.45x
650, 750, 850, 950, 1050, 1150, 1250, 1350	RT-SNN	74.5	**12.9**	**1x** (/2.55x = **-60.8%**)
	ANN	-	11.0	2.55x

Energy consumption. The theoretical energy consumption of an SNN is commonly modeled using the following formula, adopted from state-of-the-art studies [15]:

$$E = \delta \times f_r \times E_{AC} \times OP_{AC} + E_{MAC} \times OP_{MAC}, \tag{11}$$

where δ represents the number of timesteps, f_r is the firing ratio, OP_{AC} and OP_{MAC} denote the counts of accumulate (AC) and multiply-and-accumulate (MAC) operations, respectively. The energy costs per operation, E_{AC} (for AC) and E_{MAC} (for MAC), are estimated as 0.9 pJ and 4.6 pJ, respectively [12]. To evaluate the energy efficiency of RT-SNN against conventional Spiking-YOLO (SNN), we measured both mAP and energy consumption (per Eq. (11)) for tasks with period sets detailed in Table 2. Furthermore, disregarding timing guarantees, we introduce two baselines for direct comparison: SNN$_E$, configured to match RT-SNN's energy consumption (or slightly exceed it), and SNN$_A$, configured to match RT-SNN's accuracy (or be slightly lower). This matching for SNN$_E$ and SNN$_A$ was achieved by uniformly increasing their minimum timesteps (δ^{min}) across datasets.

As shown in Table 2, RT-SNN (leveraging MPR and membrane confidence) achieves higher accuracy than SNN$_E$ and lower energy



(a) Various M_{th} (b) Various γ

Figure 7: RT-SNN results with various parameters.

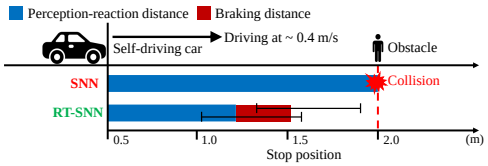


Figure 8: Emergency braking performance comparison between RT-SNN and baseline SNN on a ROS-based F1/10

consumption than SNN_A. Specifically, compared to SNN_E (iso-energy), RT-SNN improves accuracy by up to 306.7%, and compared to SNN_A (iso-accuracy), it reduces energy consumption by up to 21.9%. Notably, RT-SNN demonstrates enhanced energy efficiency and accuracy, particularly for tasks with shorter periods, owing to its ability (via MPR) to achieve high accuracy with fewer timesteps.

6.2 Comparison with existing ANN

Following the Spiking-YOLO energy model, we compare the theoretical energy consumption of an ANN on a GPU (NVIDIA V100) versus an SNN on an NPU (TrueNorth [21]). As Spiking-YOLO is derived from Tiny-YOLO [25], we use Tiny-YOLO as the base ANN model. The energy calculation is:

$$E = P_{GPU} \times T_{ANN} + P_{NPU} \times FLOPs_{SNN} \times T_{SNN}, \quad (12)$$

where P_{GPU} and P_{NPU} denote the power of the GPU and NPU, respectively. The ANN inference time is $T_{ANN} = FLOPs_{ANN}/FLOPS_{GPU}$, where $FLOPs_{ANN}$ is the total ANN FLOPs, and $FLOPS_{GPU}$ is the GPU's floating-point operations per second. $FLOPs_{SNN}$ denotes the average SNN FLOPs per timestep, and T_{SNN} is the number of inference timesteps (assuming $1ms$/timestep [21]). For a fair energy comparison, the ANN input is resized to match RT-SNN's accuracy, disregarding timing guarantees. We use hardware-specific values: P_{GPU} = 250W and P_{NPU} = 1/400W/GFLOPS. As shown in Table 3, RT-SNN achieves an energy reduction of approximately 79.0% compared to the ANN.

6.3 Ablation

Impact of various M_{th}. Fig. 7(a) illustrates the varying maximum, minimum, and average accuracy depending on M_{th} and the number of tasks. A greater M_{th} is essential for equitable resource sharing among multiple tasks, whereas a lower M_{th} is crucial to avoid underutilization of resources in scenarios with fewer tasks. A threshold M_{th} set too low (e.g., 0.01) hinders equitable resource distribution across multiple tasks by enabling the allocation of a disproportionately large share of resources to a single task during each scheduling decision, potentially diminishing the overall accuracy. An exception is observed for M_{th} = 0.01 with period set (200, 350); in this case, the high MPR reuse ratio leads to a slight increase in minimum mAP. Conversely, setting the threshold M_{th} excessively high (e.g., 0.04) can also reduce the overall accuracy due to the resultant underutilization of resources across multiple tasks. Fig. 7(a) illustrates that the optimal accuracy across various M_{th} values is achieved at an M_{th} of 0.02.

Impact of various γ. The membrane confidence estimator adjusts sensitivity to dynamics of the environment using γ in Eq. (8). Fig. 7(b) shows mAP and reuse (of membrane potential) ratio depending on γ when there are 2 and 4 tasks, respectively. In all task sets, as γ decreases, the sensitivity to the dynamics of the environment decreases, and the reuse ratio increases. High-frequency MPR does not capture the dynamics of the environment due to accumulated membrane potentials from previous frames, while low-frequency MPR does not ensure sufficient timesteps to detect objects. For example, with the task set (600, 800, 1000, 1200), RT-SNN achieves its highest mAP at 3^1.

6.4 Case study: Emergency braking

Real-world scenarios were emulated using a 1/10-scale autonomous vehicle equipped with Lidar, IMU, actuators, front/rear cameras, and a Jetson TX2 board. The front and rear cameras operate with periods of $300ms$ and $500ms$, respectively. The system architecture uses ROS nodes to communicate with a detection process, which interfaces with a scheduling thread via shared memory. The execution times for c_i^{sche}, c_i^{post}, and c_i^{conf} were measured on the Jetson TX2 (in Table 1). We assume the SNN executes one timestep in $1ms$ on a dedicated neuromorphic chip. The scenario involves the vehicle moving at $0.4m/s$ towards an obstacle $2.0m$ away, with braking triggered upon detection within $1.0m$. Each approach was evaluated over 10 trials. As illustrated in Fig. 8, the SNN failed to detect the obstacle in all 10 trials due to insufficient timesteps for reliable inference, leading to collisions. In contrast, RT-SNN, utilizing MPR to compensate for limited timesteps, successfully detected the obstacle and stopped within an average braking distance of $0.26m$, avoiding collisions in all trials.

7 Conclusion

In this paper, we presented RT-SNN, an adaptive SNN for real-time detection in AVs, resolving the conflict between high accuracy and strict timing guarantees. The proposed RT-SNN adapts its execution using a novel membrane confidence metric, guiding a co-designed runtime scheduler to dynamically adjust timesteps and apply MPR to maximize accuracy while meeting all deadlines. RT-SNN achieves up to 306.7% higher accuracy than existing SNNs at the same energy budget, and reduces energy consumption by up to 21.9% at equivalent accuracy. Against traditional ANN baselines, RT-SNN consumes up to 79.0% less energy while maintaining comparable accuracy. Validated with a Spiking-YOLO model on a ROS-based F1/10 platform, our results demonstrate that RT-SNN meets all real-time constraints.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(RS-2024-00438248, RS-2025-00520023, RS-2025-24533517, RS-2025-25424702). This work was supported by a research project with NYU (G01240629).

References

[1] Seung-Hwan Bae and Kuk-Jin Yoon. 2014. Robust online multi-object tracking based on tracklet confidence and online discriminative appearance learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1218–1225.

[2] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. 2020. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature communications* 11, 1 (2020), 3625.

[3] Marko Bertogna, Michele Cirinei, and Giuseppe Lipari. 2009. Schedulability Analysis of Global Scheduling Algorithms on Multiprocessor Platforms. *IEEE Transactions on Parallel and Distributed Systems* 20 (2009), 553–566.

[4] Anthony N Burkitt. 2006. A review of the integrate-and-fire neuron model: I. Homogeneous synaptic input. *Biological cybernetics* 95 (2006), 1–19.

[5] J. Cartucho, R. Ventura, and M. Veloso. 2018. Robust Object Recognition Through Symbiotic Deep Learning In Mobile Robots. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2336–2341.

[6] Loïc Cordone, Benoît Miramond, and Philippe Thierion. 2022. Object detection with spiking neural networks on automotive event data. In *2022 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[7] Simon Davidson and Steve B Furber. 2021. Comparison of artificial and spiking neural networks on digital hardware. *Frontiers in Neuroscience* 15 (2021), 651141.

[8] Julien Dupeyroux, Jesse J Hagenaars, Federico Paredes-Vallés, and Guido CHE de Croon. 2021. Neuromorphic control for optic-flow-based landing of MAVs using the Loihi processor. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 96–102.

[9] Yimeng Fan, Wei Zhang, Changsong Liu, Mingyang Li, and Wenrui Lu. 2024. Sfod: Spiking fusion object detector. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 17191–17200.

[10] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In *In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[11] Laurent George, Nicolas Rivierre, and Marco Spuri. 1996. *Preemptive and non-preemptive real-time uniprocessor scheduling*. Technical Report. Inria.

[12] Mark Horowitz. 2014. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*. IEEE, 10–14.

[13] Dongsung Huh and Terrence J Sejnowski. 2018. Gradient descent for spiking neural networks. *Advances in neural information processing systems* 31 (2018).

[14] Donghwa Kang, Seunghoon Lee, Hoon Sung Chwa, Seung-Hwan Bae, Chang Mook Kang, Jinkyu Lee, and Hyeongboo Baek. 2022. RT-MOT: Confidence-Aware Real-Time Scheduling Framework for Multi-Object Tracking Tasks. In *IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 318–330.

[15] Seijoon Kim, Seongsik Park, Byunggook Na, and Sungroh Yoon. 2020. Spiking-yolo: spiking neural network for energy-efficient object detection. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, Vol. 34. 11270–11277.

[16] Yuhang Li, Shikuang Deng, Xin Dong, Ruihao Gong, and Shi Gu. 2021. A free lunch from ANN: Towards efficient, accurate spiking neural networks calibration. In *International conference on machine learning*. PMLR, 6316–6325.

[17] Yuhang Li, Tamar Geller, Youngeun Kim, and Priyadarshini Panda. 2023. Seenn: towards temporal spiking early exit neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 63327–63342.

[18] Yuhang Li, Abhishek Moitra, Tamar Geller, and Priyadarshini Panda. 2023. Input-Aware Dynamic Timestep Spiking Neural Networks for Efficient In-Memory Computing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247869

[19] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *In Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 740–755.

[20] C.L. Liu and James Layland. 1973. Scheduling Algorithms for Multi-programming in A Hard-Real-Time Environment. *J. ACM* 20, 1 (1973), 46–61.

[21] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. 2014. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science* 345, 6197 (2014), 668–673.

[22] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. 2019. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine* 36, 6 (2019), 51–63.

[23] Rachmad Vidya Wicaksana Putra and Muhammad Shafique. 2023. Mantis: enabling energy-efficient autonomous mobile agents with spiking neural networks. In *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*. IEEE, 197–201.

[24] Rachmad Vidya Wicaksana Putra and Muhammad Shafique. 2023. TopSpark: a timestep optimization methodology for energy-efficient spiking neural networks on autonomous mobile agents. *arXiv preprint arXiv:2303.01826* (2023).

[25] Joseph Redmon and Ali Farhadi. 2018. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767* (2018).

[26] Qiaoyi Su, Yuhong Chou, Yifan Hu, Jianing Li, Shijie Mei, Ziyang Zhang, and Guoqi Li. 2023. Deep directly-trained spiking neural networks for object detection. In *In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 6555–6565.

[27] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, Yuan Xie, and Luping Shi. 2019. Direct training for spiking neural networks: Faster, larger, better. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, Vol. 33. 1311–1318.

[28] Nobuo Yamashita and Masao Fukushima. 2001. On the rate of convergence of the Levenberg-Marquardt method. In *Topics in Numerical Analysis: With Special Emphasis on Nonlinear Problems*. Springer, 239–249.

[29] Kashu Yamazaki, Viet-Khoa Vo-Ho, Darshan Bulsara, and Ngan Le. 2022. Spiking neural networks and their applications: A review. *Brain Sciences* 12, 7 (2022), 863.